

67-101  
10/6/18

## האוניברסיטה העברית

### בי"ס להנדסה ולמדעי המחשב

#### בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב - מואץ

67130 – מבוא למדעי המחשב - רגיל

67108 – מבוא למדעי המחשב לתלמידי תלפיות

תאריך: 17.4.2011

משך הבחינה: שלוש שעות

מועד ב' - סמסטר א' – תשע"א – 2010-2011

פרופ' דני לישצ'ינסקי, פרופ' יובל רבני, מר אלעד מזומן

#### הנחיות:

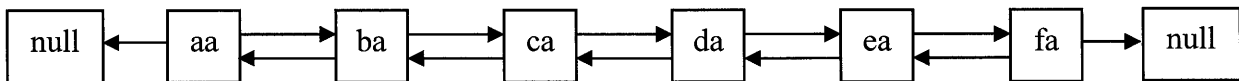
1. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
2. יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
3. יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
4. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
5. יש למחוק טיוטות באופן ברור.
6. יש להשאיר שוליים.
7. אסור להשתמש בעט אדום.
8. הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
9. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
10. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
11. אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
12. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.
13. בפתרון השאלה הינכם רשאים לבצע שימוש בשיטות שכבר מימשתם בסעיפים אחרים של אותה השאלה (אך לא בשיטות משאלות אחרות!)

67/101  
12/18/21

## שאלה 1 (33 נקודות)

### רשימה מקושרת כפולה

רשימה מקושרת כפולה (doubly linked list) היא רשימה מקושרת שבה כל חוליה מצביעה גם לחוליה הקודמת לה וגם לחוליה הבאה אחריה ברשימה, כפי שמודגם בדיאגרמה הבאה:



נתונה המחלקה הבאה לייצוג חוליה ברשימה מקושרת כפולה (שבה כל איבר מכיל מחרוזת):

```
public class Node {
    public String data;
    public Node next;
    public Node prev;
    public Node(); //default constructor
}
```

### א. (14 נקודות)

עליך לממש שיטה לא רקורסיבית:

```
public static Node swap(Node node, String data1, String data2)
```

המקבלת כקלט חוליה כלשהי ברשימה מקושרת כפולה ושתי מחרוזות, ומחפשת שתי חוליות המכילות את המחרוזות הנתונות. אם נמצאו שתי חוליות כאלה, מתבצעת החלפה בין החוליות. לבסוף מחזירה השיטה את האיבר הראשון ברשימה המעודכנת (בין אם בוצעה החלפה ובין אם לאו).

### ב. (6 נקודות)

עליך לממש שיטה לא רקורסיבית:

```
public static Node append(Node node1, Node node2)
```

המקבלת כקלט שתי חוליות שונות, כל אחת מרשימה מקושרת כפולה אחרת. השיטה מוסיפה את הרשימה השנייה לסוף הרשימה הראשונה, ומחזירה את החוליה הראשונה ברשימה המאוחדת.

### ג. (7 נקודות)

נתונה המחלקה הבאה לייצוג עץ חיפוש בינארי (המכיל מחרוזות):

```
public class BTreeNode {
```

67-101  
12/11/81

```
public String data;  
public BTNode left;  
public BTNode right;  
  
public BTNode(String data, BTNode left, BTNode  
right);  
}
```

עליך לממש שיטה רקורסיבית:

```
public static Node convertBTtoDLL(BTNode root)
```

המקבלת כקלט שורש של עץ חיפוש בינארי ומחזירה רשימה מקושרת כפולה ממוינת בסדר עולה לפי ערכי ה-  
data. שימו לב: קיים פתרון בסיבוכיות של  $O(n)$ , אך יתקבלו גם פתרונות היעילים ביותר שאינם בסיבוכיות  
 $O(n)$ .

ד. (6 נקודות)

מהי הסיבוכיות של הפתרון שהצעת בסעיף הקודם כפונקציה של  $n$ , מספר הצמתים בעץ החיפוש הבינארי? אם  
הפתרון שהוצע בעל סיבוכיות הגדולה מ- $O(n)$ , הסבירו כיצד ניתן לשפר את הפתרון כך שיעבוד ב- $O(n)$  (אין  
צורך לכתוב קוד).

67-101  
1/6"8P1

## שאלה 2 (33 נקודות)

### רקורסיה

חברכם הוא מנג'ר של קבוצת כדורגל לו ניתן תקציב בלתי מוגבל עבור רכש שחקנים. ברצונך לעזור לו להרכיב את קבוצת הכדורגל האידיאלית המונה לכל היותר  $k$  שחקנים. בשוק השחקנים ישנם  $n$  שחקנים, כאשר לכל שחקן יש אידנקס בין 0 ל  $n-1$  המייצג אותו ואותו בלבד. כמו כן, לכל שחקן משוייך מספר שלם חיובי שמציין את הכשרון שלו. הקבוצה האידיאלית היא הקבוצה שסכום כישרונות שחקניה הוא הגדול ביותר.

(א) (7 נק') תאר (אין צורך בכתיבת קוד) אלגוריתם (ניתן להשתמש באלגוריתם רקורסיבי) להרכבת הקבוצה האידיאלית (כפי שהוגדר לעיל) המונה לכל היותר  $k$  שחקנים. מה סיבוכיות זמן הריצה של אלגוריתם זה?

גם למנג'ר בעל תקציב בלתי מוגבל לא פשוט להרכיב קבוצת כדורגל. בשל בעיות אגו לא כל השחקנים מסתדרים זה עם זה. במקרה שבקבוצה ישנם שחקנים שלא מסתדרים אחד עם השני הקבוצה לא תתפקד ולכן לא ניתן להרכיב קבוצה כזאת. ניתן להניח שישנה סימטריה בין שחקנים – אם שחקן א' לא מסתדר עם שחקן ב' אז שחקן ב' לא מסתדר עם שחקן א'.

נתונה המחלקה Player אשר לה השיטות הבאות:

`public int getTalent()` מחזירה את הכישרון של אותו שחקן (ערך חיובי)

`public String toString()` מחזירה את שם השחקן

`public boolean canGetAlongWith(Player[] others)`

מחזירה אמת אם ורק אם השחקן הנוכחי יכול להסתדר עם כל השחקנים המופיעים במערך הנתון (`others`), זמן הריצה של שיטה זו הוא  $O(others.length)$ . המתודה עוברת על פני כל השחקנים במערך `others`, באם יש ערכי null היא מדלגת עליהם.

`public boolean canGetAlongWith(boolean[] others)`

מחזירה אמת אם ורק אם השחקן הנוכחי יכול להסתדר עם כל השחקנים המצויינים באמצעות מערך בוליאני בגודל  $n$ , `others[i]` מציין אם יש לבדוק עבור השחקן ה- $i$ . זמן הריצה של שיטה זו הוא  $O(n)$ .

משימתכם היא לכתוב מתודה סטטית אשר תמצא עבור חברכם המנג'ר את הקבוצה האידיאלית. בשאלה זו מותר להקצות מערכים בגודל לינארי בגודל הקלט ( $k$  ו/או  $n$ ). חתימת המתודה שעליכם לכתוב היא:

`public static void printBestTeam(Player[] players, int k)`

המערך `players` מחזיק את האובייקטים המייצגים את כל השחקנים בשוק (אורך המערך  $n$ ). על המתודה להדפיס למסך את השחקנים בקבוצה האידיאלית (סדר ההדפסה והפורמט אינם חשובים). שימו לב שיתכן מצב בו הקבוצה האידיאלית מונה פחות מ- $k$  שחקנים.

**הנחיה:** דרך אפשרית אחת לפתרון השאלה היא באמצעות העברת מערך עזר בגודל  $k$  שמכיל את הקבוצה הנוכחית, פתרון זה דומה לבעיית ה-knapsack שניתנה בשיעורי הבית, דרך אפשרית אחרת היא שימוש במערך בוליאני בגודל  $n$  אשר מציין האם השחקן ה- $i$  נמצא בקבוצה הנוכחית או לאו, פתרון זה דומה לבעיית ה-DPLL שנלמדה בכיתה. שני הפתרונות יתקבלו.  
 שימו לב שניתן להעביר כאחד הפרמטרים את הקבוצה הכי טובה שנמצאה עד כה.

(ב) (21 נק') כתבו את השיטה `printBestTeam` (ניתן להגדיר שיטות עזר נוספות). על הפיתרון לעבוד באופן

רקורסיבי, פתרונות אחרים לא יתקבלו.

ניתן להשתמש בכך שביצירת מערך אובייקטים ערך ברירת המחדל עבור האובייקטים הוא `null`.

ניתן להשתמש במתודה הבאה לצורך העתקת מערכים:

`System.arraycopy(Object src, int srcPos, Object dest, int destPos, int length)`

(ג) (3 נק') תנו חסם עליון לזמן הריצה של התוכנית, השתמשו ב- $k$  וב- $n$  (מספר השחקנים הכללי).

(ד) (2 נק') אם ניתן להניח כי  $k \ll n$  ( $k$  קטן בסדרי גודל מ- $n$ ). האם היית משנה את פתרוןך? הסבר מדוע כן או

לא. רמז: חשבו על ההבדלים בין שני הפתרונות המוצעים בהנחיה.

67.101  
12/10/19

### שאלה 3 (33 נקודות)

עליך לענות על כל הסעיפים.

קרא את קטע הקוד הבא:

```
public class LinkedList {
    public static class Node{
        private int _a;
        private Node _b;
        public Node(int a){
            _a = a;
        }
        public Node(int a, Node b){
            _a = a;
            _b = b;
        }
        public int getA(){
            return _a;
        }
    }
    private Node _c;
    private Node _d;
    public LinkedList(){
        _c = null;
        _d = null;
    }
    public LinkedList(Node n){
        _c = n;
        _d = _c;
        if (_d!=null){
            while (_d._b!=null){
                _d = _d._b;
            }
        }
    }
    public void foo(int num){
        if (_c==null){
            _c = new Node(num);
            _d = _c;
        }
        else{
            _d._b = new Node(num);
            _d = _d._b;
        }
    }
    public LinkedList foo1(){
        return foo1(this);
    }
    private LinkedList foo1(LinkedList list){
        Node n = list._c;
```

67.101  
2/k"8PA

```

    if (n==null){
        return list;
    }
    int c = n.getA();
    LinkedList list1 = new LinkedList();
    LinkedList list2 = new LinkedList();
    Node temp = n._b;
    while (temp!=null){
        int v = temp.getA();
        if (v<c)
            list1.foo(v);
        else
            list2.foo(v);
        temp = temp._b;
    }
    list1 = foo1(list1);
    list1.foo(c);
    list2 = foo1(list2);
    list1.foo2(list2);
    return list1;
}
public void foo2(LinkedList other){
    if (other._c==null)
        return;
    if (_c==null)
        _c = other._c;
    _d._b = other._c;
    _d = other._d;
}
public String toString(){....}; //assume the method is
//implemented as described below
public static void main(String[] args){
    LinkedList list = new LinkedList();
    list.foo(12);
    list.foo(51);
    list.foo(10);
    list.foo(3);
    System.out.println(list);
    list = list.foo1();
    System.out.println(list);
}
}

```

(א) 2 נקודות

מהם תפקידי המשתנים  $a$  ו  $b$  במחלקה `Node`?

(ב) 2 נקודות

מהו התפקיד של המשתנים  $c$  ו  $d$  במחלקה `LinkedList`?

(ג) 3 נקודות

מה מבצעת השיטה `foo`?

(ד) 4 נקודות

מה מבצעת השיטה `foo2`?

67.101  
12/16" 8P1

(ה) (7 נקודות)

מה מבצעת השיטה foo1? הסבר את האלגוריתם בו היא משתמשת.

(ו) (5 נקודות)

מה יודפס למסך בהרצה של main? הנח שקיימת שיטת toString שמדפיסה את אברי הרשימה לפי הסדר משמאל (תחילת הרשימה) לימין (סוף הרשימה).

(ז) (5 נקודות)

מהי סיבוכיות זמן הריצה של foo1 במקרה הטוב/הגרוע ביותר?

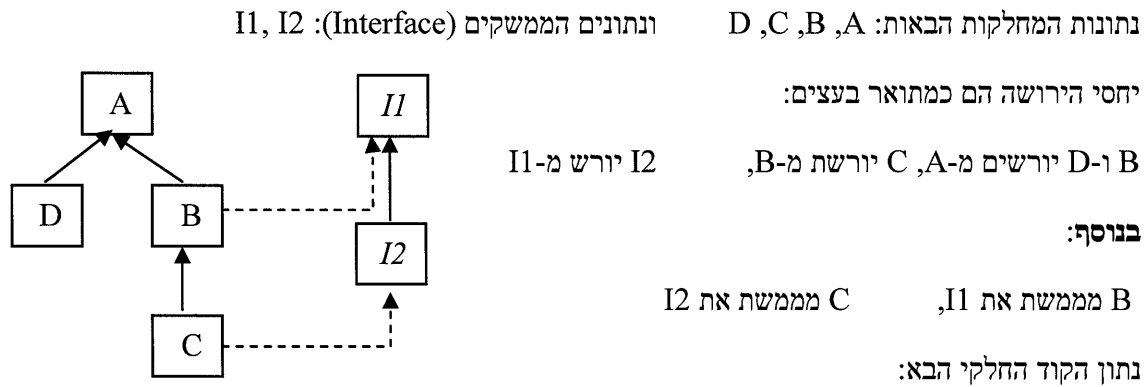
(ח) (5 נקודות)

תן דוגמא לקלטים בעלי חמישה פריטים לפחות עבור המקרה הטוב והגרוע ביותר בשביל השיטה foo1.

67-101  
2/10/89

## שאלה 4 (33 נקודות)

ירושלם, ממשקים ופולימורפיזם



```
public class A{
    public void f1() {System.out.println("I'm A-type creature");}
}

public class B {
    public void f1() {System.out.println("I'm B-type creature");}
    public void f2(I2 i) {i.g2(this);}
}

public class C{
    public void f1() {System.out.println("I'm C-type creature");}
    public void f2(I2 i) {}
    public void g1() {}
    public void g2(I1 i) {i.f1();}
}

public class D {}

public interface I1{
    public void f1();
    public void f2(I2 i);
}

public interface I2{
    public void g1();
}
```

67.101  
2/6"8 P11

```
public void g2(I1 i);
```

```
}
```

א. השלימו/תקנו את הקוד כך שיבטא את יחסי הירושה והמימוש שתוארו לעיל (4 נקודות)

ב. נתון הקוד הבא. לגבי כל אחת מהשורות הממוספרות – האם היא מתקמפלת, אם כן – האם היא גורמת ל-Exception? (9 נקודות)

```
A a = new A(); B b = new B(); C c = new C();
```

```
D d = new D(); I1 i1; I2 i2;
```

```
1. i1 = b;
```

```
2. i1 = c;
```

```
3. i1 = (I1)a;
```

```
4. i2 = b;
```

```
5. i2 = (I2)b;
```

```
6. i2 = c;
```

```
7. i2 = a;
```

```
8. i2 = d;
```

```
9. i2 = (I2)d;
```

```
10. i1 = c;
```

```
11. i2 = i1;
```

```
12. i2 = (I2)i1;
```

```
13. i1 = i2;
```

```
A a1; B b1; C c1; D d1;
```

```
14. a1 = b;
```

```
15. b1 = a;
```

```
16. b1 = (B)d;
```

```
17. c1 = (C)a;
```

```
18. d1 = a;
```

67.101  
12/16"8P1

ג. נתון קטע הקוד הבא. לגבי כל שורה בו – האם היא מתקמפלת, אם לא – האם אפשר לתקן אותה כך שתתקמפל ולא תגרום לשגיאה בזמן ריצה? מה יקרה בזמן ריצה בכל אחת מהשורות ומה יודפס? (9 נקודות)

```
1. I1 []i1Array = new I1[3];
2. i1Array[0] = new A();
3. i1Array[1] = new B();
4. i1Array[2] = new C();
5. for (int i = 0; i < i1Array.length; ++i){
6.         if (null != i1Array[i]){
7.                 i1Array[i].f1();
8.         }
9. }
```

ד. מה יהיה הפלט של קטע הקוד הבא (6 נקודות)

```
B bb = new B();
C cc = new C();
bb.f2(cc);
```

ה. מה יהיה הפלט של קטע הקוד הבא (5 נקודות)

```
A []aArray = new A[6];
aArray[0] = new A();
aArray[1] = new B();
aArray[2] = new C();
aArray[3] = new D();
for (int i = 0; i < aArray.length; ++i){
    if (null != aArray[i]){
        aArray[i].f1();
    }
}
```